TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 16

Node.js, HeroUI app init, tour & first component

Phase 3 — AI-Assisted Development · Weeks 13–18 · Modules 05–06 + Capstone

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. Module 06 — HeroUI Frontend: overview
2. 06.00 Prerequisites Check
3. 06.01 Install Node.js with nvm
4. 06.02 Init the HeroUI App
5. 06.03 Tour + First Component

Module 06 — HeroUI Frontend

What you'll be able to do after this module

- Install Node.js with nvm and confirm `node`, `npm`, and `npx` resolve to nvm-managed paths
- Initialize a HeroUI starter project with Next.js and Tailwind pre-configured and run its dev server with hot reload
- Build interactive pages using HeroUI components such as `Button`, `Input`, `Card`, and `Checkbox`, applying the `"use client"` directive where hooks are used
- Enable CORS in the FastAPI backend with `CORSMiddleware` so the browser-based frontend can call it
- Fetch items from `http://localhost:8000/items` and render them as HeroUI Cards with loading, error, and empty states
- Build a form that POSTs new items to the API and refreshes the displayed list inline without a page reload
- Use Continue.dev chat, inline edit, and agent mode to extend the frontend, reviewing every proposed diff before accepting
- Ship a frontend change through a branch, commit, and merge or GitHub PR workflow

Lessons and objectives

#	Lesson	Learning objective
00	Prerequisites Check	Verify that the FastAPI Compose stack, Continue.dev connection, and GitHub remote from earlier modules still work and that no conflicting system Node.js installation is present.
01	Install Node.js with nvm	Install nvm and the latest LTS release of Node.js, set it as the default, and confirm `node`, `npm`, and `npx` run from nvm-managed paths under your home directory.
02	Init the HeroUI App	Scaffold a Next.js + HeroUI starter project with `npx heroui-cli@latest init`, install its dependencies, and run the dev server so the starter page loads on port 3000.
03	Tour + First Component	Build an interactive homepage from HeroUI `Button`, `Card`, and `Input` components and explain when a Next.js page needs the `"use client"` directive to use React hooks.
04	CORS + Fetch Items	Add `CORSMiddleware` to the FastAPI backend and fetch the items list from the browser, rendering each item as a HeroUI Card with loading, error, and empty states.
05	Create-Item Form	Build a HeroUI form that POSTs new items to FastAPI, converts input strings to the types the API expects, surfaces server validation errors, and reloads the list inline on success.
06	Extend with Continue.dev	Use Continue.dev agent mode on a git feature branch to add a Delete button with a confirmation modal, reviewing every proposed diff and recognizing when to abort a failing agent loop.
07	Final Checkpoint	Ship a client-side, case-insensitive search filter for the items page end-to-end using Continue.dev, verify it in the browser, and commit and merge the change.

Each lesson is one learning object with a paired lab. Work in order — completing a lab unlocks the next lesson. The module ends with a **Check on Learning** quiz and a hands-on final checkpoint.

Learning objective: Verify that the FastAPI Compose stack, Continue.dev connection, and GitHub remote from earlier modules still work and that no conflicting system Node.js installation is present.

Lesson 00: Prerequisites Check

By the end, you will have confirmed Modules 03–05 still work and that you don't already have Node.js installed in a conflicting way.

~5 minutes

---

1. Open VS Code connected to your dev environment. Open the integrated terminal.

2. Confirm the FastAPI Compose stack still starts.

```
cd ~/docker-course/fastapi-app
docker compose up -d
sleep 3
curl -s http://localhost:8000 | python3 -m json.tool
```

You should see `{"hello":"fastapi"}` (or whatever you renamed it).

3. Confirm Continue.dev is wired to the shared vLLM endpoint (Llama). Open the Continue side bar and send the message:

```
Reply with the single word: ready
```

You should get `ready` back within ~10 seconds. If not, see Module 05 lesson 04.

4. Confirm GitHub is reachable.

```
git -C ~/docker-course/fastapi-app remote -v
```

Should show your GitHub origin URL.

5. Check whether Node.js is already installed (we want to use **nvm** to manage versions cleanly — a system-installed Node can interfere).

```
which node npm 2>&1
```

6. If you see `/usr/bin/node` or similar, you have a system Node. That's not a blocker but flag it to the instructor — we'll prefer the nvm-managed one in lesson 01.

7. If you see `which: no node in (...)`, perfect — clean slate.

8. Log this check.

```
echo "Module 06 prereq check: passed $(date -Iseconds)" >> ~/docker-course/notes/notes.txt
```

Watch out

- The FastAPI stack from Module 03 must be running for lesson 04 onward. Easiest: leave it up for the whole module.
- The shared vLLM endpoint from Module 05 must be reachable so Continue.dev works.

Checkpoint

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000
```

Should print `200`.

```
docker compose -f ~/docker-course/fastapi-app/docker-compose.yml ps --status=running | tail -n +2 | wc -l
```

Should print `2` (api + db running).

Next up: 01-install-nodejs.md — Install Node.js with nvm

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Install nvm and the latest LTS release of Node.js, set it as the default, and confirm `node`, `npm`, and `npx` run from nvm-managed paths under your home directory.

Lesson 01: Install Node.js with nvm

By the end, you will have nvm installed, the latest LTS version of Node.js active, and `node` / `npm` / `npx` on your PATH.

~10 minutes of typing

> **nvm** (Node Version Manager) lets you install and switch between Node.js versions without sudo. It's the standard way to manage Node on a developer machine.

---

1. Make sure you're in your home directory.

```
cd ~
```

2. Install nvm using the official installer.

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

You should see output ending with:

```
=> nvm source string already in /home/your-user/.bashrc
=> Close and reopen your terminal to start using nvm
```

3. Either close+reopen the terminal or source your shell config now.

```
source ~/.bashrc
```

4. Confirm nvm is loaded.

```
command -v nvm
```

```
nvm
```

(Expected output is the literal word `nvm` — it's a shell function, not a binary.)

5. See what Node versions nvm can install.

```
nvm ls-remote --lts | tail -n 10
```

You'll see a list of LTS versions.

6. Install the latest LTS.

```
nvm install --lts
```

You should see lines about downloading and installing, ending with:

```
Now using node v20.x.x (npm v10.x.x)
```

7. Mark it as the default for new shells.

```
nvm alias default lts/*
```

8. Confirm Node and npm work.

```
node --version
npm --version
npx --version
```

You should see three version numbers (e.g. `v20.11.0`, `10.2.4`, `10.2.4`).

9. See where they live (under your home, not system paths).

```
which node npm npx
```

```
/home/your-user/.nvm/versions/node/v20.11.0/bin/node
/home/your-user/.nvm/versions/node/v20.11.0/bin/npm
/home/your-user/.nvm/versions/node/v20.11.0/bin/npx
```

10. Run a one-liner to confirm Node is working end to end.

```
node -e "console.log('node ok:', process.version)"
```

```
node ok: v20.11.0
```

11. Configure npm to use a sensible cache location (optional but tidy).

```
npm config set fund false
npm config set audit false
```

(Disables nag messages on every install.)

12. Log it.

```
echo "Node installed: $(node --version), npm $(npm --version)" >> ~/docker-course/notes/notes.txt
```

Watch out

- **`nvm` is a shell function, not a binary.** `which nvm` returns nothing — that's correct. Use `command -v nvm` to verify it exists.
- **Every new terminal sources `~/.bashrc`** which sources nvm. If a fresh terminal can't find Node, check that `~/.bashrc` has the nvm block (it should — the installer added it).
- **Don't `sudo npm install -g`.** When npm wants to write to a system path, that's a sign your nvm setup isn't active. Fix nvm; don't sudo around it.
- **WSL users:** the same nvm setup works in WSL exactly like native Linux. Don't try to use a Windows-side Node.

Checkpoint

```
node --version && npm --version && npx --version
```

Should print three version numbers.

```
which node | grep -q '\.nvm' && echo "OK: nvm-managed"
```

Should print `OK: nvm-managed`.

```
node -e "console.log(2 + 2)"
```

Should print `4`.

Next up: 02-init-heroui-app.md — Init the HeroUI starter project

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Scaffold a Next.js + HeroUI starter project with `npx heroui-cli@latest init`, install its dependencies, and run the dev server so the starter page loads on port 3000.

Lesson 02: Init the HeroUI App

By the end, you will have a Next.js + HeroUI starter project at `~/docker-course/fastapi-frontend/` with all dependencies installed and a dev server running on port 3000.

~15 minutes of typing + waiting for `npm install`

> **HeroUI** (formerly NextUI) is a React component library: prebuilt buttons, inputs, cards, modals — all themed and accessible. Pairs cleanly with Next.js + Tailwind. Their CLI scaffolds a starter that has everything wired up.

---

1. Move into the course directory.

```
cd ~/docker-course
```

2. Confirm it's there.

```
pwd
```

```
/home/your-user/docker-course
```

3. Run the HeroUI CLI to scaffold a new app. This downloads the CLI on the fly via npx (no global install needed).

```
npx heroui-cli@latest init
```

4. The CLI prompts you. Answer:

- **Project name:** `fastapi-frontend`
- **Template:** pick the **Next.js** option (or "App" — the simplest one).
- **Package manager: npm**.
- **Install dependencies now: Yes**.

5. The CLI creates the directory and runs `npm install`. This takes 1–3 minutes (~150 MB of dependencies). Let it finish.

6. Move into the new project.

```
cd fastapi-frontend
```

7. Look at what got created.

```
ls -la
```

You should see:

```
.eslintrc.json
.gitignore
README.md
app/                    (Next.js App Router pages live here)
components/             (your custom components)
config/
node_modules/           (DON'T look inside — 50,000+ files)
package.json
package-lock.json
public/
styles/
tailwind.config.js
tsconfig.json
```

8. Read `package.json` to see what's installed.

```
cat package.json | python3 -m json.tool | head -n 30
```

You'll see `next`, `react`, `@heroui/react`, `tailwindcss`, `framer-motion` (HeroUI's animation peer dep) and friends.

9. Start the dev server.

```
npm run dev
```

You should see:

```
> next dev

  ▲ Next.js 14.x.x
  - Local:        http://localhost:3000
  - Environments: .env.local
 ✓ Ready in 1.2s
```

10. Leave that terminal running. Open a **second** integrated terminal in VS Code.

11. Open `http://localhost:3000` in your browser.

- **WSL students:** Windows browser, `http://localhost:3000`.
- **SSH students:** VS Code's "Ports" tab should auto-forward 3000. Click the globe icon.

12. You should see a HeroUI starter page — usually something like a "Welcome to HeroUI" landing page with example components.

Manual fallback (if the CLI fails)

If `heroui-cli` fails for any reason, do this manually:

Manual A. Create a Next.js app:

```
cd ~/docker-course
npx create-next-app@latest fastapi-frontend --typescript --tailwind --app --eslint
--src-dir=false --import-alias='@/*'
cd fastapi-frontend
```

Manual B. Install HeroUI:

```
npm install @heroui/react framer-motion
```

Manual C. Configure Tailwind for HeroUI. Open `tailwind.config.js` and replace the `content` array and add the plugin per https://heroui.com/docs/guide/installation. (The CLI did this automatically; here you do it by hand.)

Manual D. Wrap your app with `HeroUIProvider`. Open `app/layout.tsx` (or `app/providers.tsx` if using a separate file) and wrap `{children}` in `<HeroUIProvider>`.

The CLI path is faster — only fall back to manual if necessary.

---

13. Back in the dev terminal, you should see Next.js compile messages every time you save a file. Leave it running.

14. Initialize a git repo for the frontend (it's a separate repo from your FastAPI backend).

```
git init
git add .
git commit -m "Initial commit: HeroUI Next.js starter"
```

15. Log it.

```
echo "HeroUI app initialized at ~/docker-course/fastapi-frontend" >> ~/docker-course/notes/notes.txt
```

Watch out

- `node_modules/` **is huge.** Never commit it — the CLI's `.gitignore` already excludes it. Confirm: `grep node_modules .gitignore`.
- **Port 3000 in use.** If you already have something on 3000, Next.js will pick 3001 automatically. Watch the URL it prints.
- **Don't run** `npm install` **from the home directory.** Always from the project directory (`~/docker-course/fastapi-frontend/`). Otherwise you create a `node_modules/` in the wrong place.
- **The CLI version pins matter.** HeroUI is moving fast — major versions change props/imports. The starter will pin a compatible set; don't manually upgrade individual packages mid-module.

Checkpoint

```
cd ~/docker-course/fastapi-frontend && ls package.json
```

Should print `package.json`.

```
grep -q '"@heroui/react"' ~/docker-course/fastapi-frontend/package.json && echo "OK: HeroUI installed"
```

Should print `OK: HeroUI installed`.

```
curl -s http://localhost:3000 | head -c 200
```

Should print HTML (something starting with `<!DOCTYPE html>` or similar).

In the browser, `http://localhost:3000` shows the HeroUI starter page.

Next up: 03-tour-and-first-component.md — Tour the app and add your first HeroUI component

> **Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Build an interactive homepage from HeroUI `Button`, `Card`, and `Input` components and explain when a Next.js page needs the `"use client"` directive to use React hooks.

Lesson 03: Tour + First Component

By the end, you will have changed the homepage to display a single HeroUI Button, watched it hot-reload, and seen what `"use client"` does.

~15 minutes of typing

---

1. Make sure the dev server is still running. If not:

```
cd ~/docker-course/fastapi-frontend
npm run dev
```

2. In VS Code, open `~/docker-course/fastapi-frontend` as a folder (`File → Open Folder…`).

Tour

3. Look at the structure in the explorer:

```
app/
├── layout.tsx       ← root layout, wraps every page
├── page.tsx         ← the / route
├── providers.tsx    ← HeroUIProvider lives here
├── globals.css      ← Tailwind directives
components/
config/
public/
styles/
tailwind.config.js
```

4. Open `app/page.tsx`. This is what renders at `http://localhost:3000`. It probably has a `<Hero>` or similar landing-page setup.

5. Open `app/layout.tsx`. Note the `<Providers>` component wrapping `{children}` — that's where HeroUI's theme and context get attached.

First change

6. Replace the contents of `app/page.tsx` with a minimal page:

```
import { Button } from "@heroui/react";

export default function Home() {
  return (
    <main className="flex min-h-screen flex-col items-center justify-center gap-4
p-8">
      <h1 className="text-3xl font-bold">FastAPI + HeroUI</h1>
      <Button color="primary">Click me</Button>
    </main>
  );
}
```

7. Save. Watch the browser tab — Next.js hot-reloads. The page should now show your heading and a primary-color HeroUI button.

8. Click the button. Nothing happens — we haven't wired an `onClick` yet. That's intentional.

9. Look at the dev-server terminal. You should see:

```
 ✓ Compiled / in 200ms
```

"use client" — the moment

10. Try adding interactivity. Update `app/page.tsx` to use `useState`:

```
import { Button } from "@heroui/react";
import { useState } from "react";

export default function Home() {
  const [count, setCount] = useState(0);

  return (
    <main className="flex min-h-screen flex-col items-center justify-center gap-4
p-8">
      <h1 className="text-3xl font-bold">FastAPI + HeroUI</h1>
      <Button color="primary" onPress={() => setCount(count + 1)}>
        Clicked {count} times
      </Button>
    </main>
  );
}
```

11. Save. **The browser will show an error** — something like:

```
You're importing a component that needs `useState`. It only works in a Client
Component but none of its parents are marked with "use client", so they're Server
Components by default.
```

12. This is the most important Next.js lesson. Pages and components in `app/` default to **server components** — they render on the server and ship as HTML. Anything with hooks (`useState`, `useEffect`, browser APIs) needs to be a **client component**, which means a `"use client"` directive at the top.

13. Add `"use client";` as the very first line of `app/page.tsx`:

```
"use client";

import { Button } from "@heroui/react";
import { useState } from "react";
```

14. Save. The error disappears, and the button now counts up when clicked.

15. Click it a few times. The number increments. **You have a working interactive React app.**

Try a few more components

16. Replace `app/page.tsx` with a slightly richer page:

```
"use client";

import { Button, Card, CardBody, CardHeader, Input } from "@heroui/react";
import { useState } from "react";

export default function Home() {
  const [name, setName] = useState("");

  return (
    <main className="flex min-h-screen flex-col items-center justify-center gap-4
p-8">
      <h1 className="text-3xl font-bold">FastAPI + HeroUI</h1>

      <Card className="w-96">
        <CardHeader>
          <h2 className="text-lg font-semibold">Greeter</h2>
        </CardHeader>
        <CardBody className="flex flex-col gap-4">
          <Input
            label="Your name"
            value={name}
            onValueChange={setName}
          />
          <Button color="primary" isDisabled={!name}>
            Hello, {name || "stranger"}
          </Button>
        </CardBody>
      </Card>
    </main>
  );
}
```

17. Save. The page now shows a HeroUI Card with an Input and a Button that updates as you type.

18. Type your name in the input — the button text changes live.

19. Notice how the Button is disabled while the input is empty (`isDisabled={!name}`).

Use Continue.dev for a tweak

20. Open the Continue chat panel (Module 05). Highlight the entire `Home` component in `app/page.tsx` .

21. Press **Ctrl+I**. Type:

```
Add a Reset button next to the greeting button that clears the name input.
```

22. Read the diff before accepting. The model should add a second Button with an `onPress` that calls `setName("")`.

23. Accept. Type a name, then click Reset. The input clears.

24. This is the agentic-coding loop on a frontend codebase. Same as Module 05's drills, just against TSX instead of Python.

Commit

25. From a second terminal:

```
cd ~/docker-course/fastapi-frontend
git add app/page.tsx
git commit -m "First HeroUI page: greeter with Card, Input, Button"
```

26. Log it.

```
echo "HeroUI tour: Card + Input + Button + use client + Continue.dev tweak" >>
~/docker-course/notes/notes.txt
```

Watch out

- **`"use client"` must be on line 1**, before any imports. (Comments above it are fine.)
- **Server vs client components is the #1 Next.js confusion.** Hooks → client. No hooks → server (and that's faster). Don't mark every component `"use client"` reflexively — it loses the SSR benefit.
- **HeroUI uses `onPress`, not `onClick`.** They use react-aria internally. `onClick` may work but `onPress` is the documented prop.
- **Tailwind classes are space-separated strings.** Quotes around the whole `className`, no commas. `"flex flex-col gap-4 p-8"` not `"flex, flex-col, gap-4, p-8"`.

Checkpoint

```
grep '"use client"' app/page.tsx
```

Should match.

```
grep -E "Card|Input|Button" app/page.tsx | head -n 3
```

Should show HeroUI imports/usages.

In the browser at `http://localhost:3000`:

- A heading "FastAPI + HeroUI"
- A Card with an Input and Buttons
- The Reset button works

Next up: 04-cors-and-fetch-items.md — Connect the frontend to FastAPI

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.